New Project Setup — Reusable Checklist

Every step performed, in order, so this can be repeated for any future project.

Part A — Create the Project Folder

Step 1: Pick a location for all your projects

Create one main folder where all your coding projects will live (e.g. a 'Projects' folder on any drive). Then create a folder inside it for this specific project.

Why: Keeps every project in one predictable place instead of scattered across Desktop/Downloads.

Step 2: Install Git (first time on a new laptop only)

Download from git-scm.com/download/win, run the installer accepting all default options, then restart VS Code.

Why: Git is the tool that saves versions of your work and lets you back it up to GitHub.

Step 3: Verify Git installed

Open VS Code, open a terminal (Terminal menu → New Terminal), and check the version.

```
git --version
```

Why: Confirms the install actually worked before relying on it.

Step 4: Set your Git identity (first time only, not per-project)

Tell Git your name and email — this only needs doing once per laptop, ever.

```
git config --global user.name "Your Name" git config --global user.email "you@example.com"
```

Why: Every saved snapshot (commit) gets tagged with this, and it's what links your work to your GitHub profile.

Step 5: Verify identity was saved

Check both values printed back correctly.

```
git config --global user.name git config --global user.email
```

Step 6: Navigate into your project folder

Move your terminal into the exact folder for this project.

```
cd "FULL_PATH_TO_YOUR_PROJECT_FOLDER"
```

Why: All following commands assume you're standing inside the project folder.

Step 7: Create the folder structure

Create all subfolders needed for the project in one command (adjust names to fit the project type).

```
mkdir data\raw, data\cleaned, data\processed, data\external, notebooks, src\preprocessing, src\utils
```

Why: A clean, consistent structure from day one — easy to navigate, easy for others (professors, recruiters) to understand.

Step 8: Open the folder visually in VS Code

Open the project as a workspace so you can see/edit files in the sidebar instead of only the terminal.

```
code .
```

Part B — Add Starter Files

Step 9: Create requirements.txt

A plain text file listing every library the project needs, one per line.

Why: Lets anyone (including future you) install everything with one command: pip install -r requirements.txt

Step 10: Create .gitignore

A plain text file listing files/folders Git should never track (cache files, large data files, secret keys).

Why: Without this, Git tries to save junk files and even accidental secrets — created BEFORE the first commit so it works cleanly from the start.

Step 11: Create README.md

A markdown file explaining what the project is, its folder structure, and how to run it.

Why: The first thing anyone sees when they open your GitHub repo — sets the impression for recruiters/professors.

Part C — Start Version Control (Git)

Step 12: Initialize Git in the folder

Turn this folder into a Git repository so it can start tracking changes.

```
git init
```

Why: Without this, Git ignores the folder completely. Do this once per project, at the very start.

Step 13: Check status

See what Git currently notices (new/changed files).

```
git status
```

Step 14: Stage your files

Tell Git which changes to include in the next saved snapshot.

```
git add .
```

Why: The dot means 'everything in this folder' (minus what .gitignore excludes).

Step 15: Commit (save a snapshot)

Save a permanent point-in-time version with a short message describing what changed.

```
git commit -m "Initial project structure"
```

Why: This is your first real save point. Repeat this after every meaningful chunk of work, with a fresh message each time.

Part D — Connect to GitHub

Step 16: Create a new repository on GitHub.com

Go to github.com → '+' icon → New repository → name it → keep Public → do NOT check 'Add README/.gitignore/license' (since these already exist locally) → Create repository.

Why: GitHub is your online backup AND your shareable portfolio link.

Step 17: Link your local folder to the GitHub repo

Copy the repository URL GitHub shows you, then connect it.

```
git remote add origin https://github.com/yourusername/your-repo-name.git
```

Why: This tells Git where 'online' actually is. The nickname 'origin' is the standard default name.

Step 18: Verify the connection

Confirm the URL is linked correctly for both fetch and push.

```
git remote -v
```

Step 19: Push your first commit

Upload your saved snapshot(s) to GitHub for the first time.

```
git push -u origin master
```

Why: The -u flag remembers this link, so every future push can just be 'git push' with nothing extra.

Part E — Repeat This Every Work Session From Now On

Once setup (Parts A-D) is done, every future work session follows this same 3-command cycle:

```
1 git add .
2 git commit -m "short description of what changed"
3 git push
```

Tip: commit small, often, and with clear messages. A long history of small honest commits is more valuable on a resume/portfolio than one giant commit at the end.

Reusable setup checklist — keep this for every new coding project.